



Ağ Modeli ve Uygulama Erişimi

Kubernetes ve Konteyner Dünyası



Bu konuda

- Pod, node ve Service IP'lerini ayıran ağ modeli
- Service, EndpointSlice, portlar ve cluster içi DNS keşfi
- Service türleri, trafik yakınlığı ve erişim yöntemleri
- Ingress ile Gateway API'nin sorumluluk sınırları
- NetworkPolicy seçicileri ve yön bazlı izolasyon

Kubernetes ađ modeli





Üç IP, üç ayrı görev

Bir Kubernetes cluster'ında birbirinden farklı üç tane IP yönlendirme katmanı bulunur:

- **Pod IP'si**
- **Service IP'si**
- **Node IP'si**

Bu üç yönlendirme katmanının adres aralıkları birbiriyle çakışmayacak biçimde ayrılır^[2]



Pod IP'si

- Her Pod için benzersiz bir IP adresi; aynı Pod'daki konteynerler adresi ve port alanını paylaşır^[1]
- Adresi CNI eklentisi atar^[2]
- Pod yeniden oluşturulunca değişebilir
- **Pod'ların Pod IP'leri aracılığıyla node ayrımlarından bağımsız ve NAT olmadan doğrudan haberleşme garantisi vardır^[1]**



CNI eklentisi

- Pod IP yönlendirme katmanını gerçekleştiren yazılım
- Her node'da konteyner runtime'lar tarafından kullanılır



Service

- Etiketlerle seçilen bir Pod grubunun Pod IP'lerinin belirlenmiş portlarına yönlendirme yapan sanal bir IP yönlendirme katmanı ve DNS çözümlemesi kombinasyonu ^[2,3]
- Yönlendirilen Pod'ların fiziksel arayüzlerine bağlı bir Pod adresi değildir; service proxy ilişkilendirilmiş Pod'lara yönlendirme yapar ^[5]
- CNI eklentisinin farklı bir gerçekleştirim sağlamaması durumunda `kube-proxy` Servis - Pod IP arasındaki yönlendirme için node'larda kurallar oluşturur



Service ve EndpointSlice

- Service'in yönlendirme yapması gereken Pod IP'leri, cluster tarafından **EndpointSlice** nesneleri tarafından sağlanır:

`Service selector` → eşleşen Pod'lar → `EndpointSlice` içindeki `IP:port` ve koşullar^[3,4]

- **EndpointSlice** nesneleri control plane tarafından oluşturulur, **fakat paket yönlendirmesi sürecinin kendisinde kullanılmaz:**

`İstemci` → (opsiyonel olarak) `DNS` → `Service IP:port` → `Service yönlendirme katmanı` (Service proxy aracılığıyla) → `Pod IP:Port` ^[4,5,6]

- Pod eğer Ready durumu belirtmiyorsa ilgili EndpointSlice nesnesi içinde kendisine işaret eden girdinin `ready` property'si `false` olur ve Service trafiğine hazır kabul edilmez^[4]

- `publishNotReadyAddresses: true` ile hazır olmayan adresler EndpointSlice içine `ready: true` olarak işlenebilir^[4]



Service nesneleri

```
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector:
    app: web
  ports:
  - name: http
    protocol: TCP
    port: 80
    targetPort: 8080
```

- `port`, istemcilerin Service üzerinde kullandığı porttur; `targetPort`, seçilen Pod'daki hedef porttur^[3]
- `targetPort` yazılmazsa yerine `port` değeri kullanılır; sayı yerine konteyner portunun adı verilebilir^[3]
- Adlandırılmış hedef port, farklı Pod sürümlerinin aynı port adını farklı sayılara bağlamasına izin verir^[3]
- `nodePort` yalnızca `NodePort` kullanan bir Service türünün node üzerindeki dış portudur^[3]



Service türleri

Tür	Sağladığı erişim	Önemli sınırlama
ClusterIP	Service IP adres aralığından kararlı sanal IP; varsayılan tür	Cluster dışına doğrudan açılmaz ^[3]
NodePort	Her node'da aynı statik port; dinlenen node adresleri proxy uygulaması ve <code>nodePortAddresses</code> ayarına bağlıdır ^[3,5]	Varsayılan aralık <code>30000–32767</code> ; gerçek erişim node rotası ve güvenlik duvarına bağlıdır ^[3]
LoadBalancer	Destekleyen uygulama veya bulut sağlayıcı üzerinden dış yük dengeleyici	Uygulama yoksa dış IP atanmayabilir ve <code>EXTERNAL-IP</code> beklemede kalabilir ^[3]
ExternalName	Service adını bir dış DNS adına CNAME olarak eşler	Proxy ve endpoint yoktur; HTTP <code>Host</code> ve TLS sertifika adı uyumsuzluğu doğabilir ^[3]



kubectl expose

Çeşitli iş yükü türlerine işaret eden Service oluşturmayı sağlar.

```
kubectl expose deployment nginx
```

- Service'in açması istenen port ve ilgili iş yükündeki açık olan konteyner portu `--port` ve `--target-port` ile belirtilebilir
- İstenen servis tipi `--type` ile belirtilebilir



Cluster DNS'i: Kısa Ad, Namespace ve FQDN

- Kubernetes, Service keşfi için DNS kayıtları yayımlar; yaygın cluster DNS uygulaması **CoreDNS**'tir^[1,3,6]
- DNS uygulaması namespace temelli olmak üzere farklı alan adı seviyelerinde çözümleme sağlar:

Yapılan sorgular	shop içindeki istemciden anlamı
<code>web</code>	İstemcinin kendi namespace'i olan <code>shop</code> içinde aranır
<code>web.payments</code>	Farklı namespace açıkça belirtilir, <code>payments</code> namespace'i içindeki <code>web</code> servisi aranır
<code>web.payments.svc</code>	Bir üstteki ile aynı
<code>web.payments.svc.cluster.local</code>	Varsayılan cluster alan adı için tam nitelikli alan adı (FQDN)
<code>_http._tcp.web.payments.svc.cluster.local</code>	<code>http</code> adlı Service portunun SRV kaydı

- Pod'un arama listesi normalde kendi namespace'ini, `svc` alanını ve cluster domain'ini içerir; bu yüzden aynı namespace'te kısa ad yeterlidir^[6]
- `cluster.local` yaygın varsayımdır, fakat cluster yöneticisi farklı bir domain seçebilir^[6]



Headless ve Selector'sız Service

- **Headless Service:** `clusterIP: None`; sanal IP ve Service proxy yük dengelemesi yoktur. DNS, EndpointSlice içindeki IP'leri doğrudan döndürür^[3]
- **Selector'sız Service:** control plane EndpointSlice üretmez; kullanıcı veya ayrı bir controller endpoint durumunu yönetir^[3,4]
- İki özellik bağımsızdır: bir Service headless olup selector kullanabilir veya ClusterIP sahibi olup selector'sız olabilir



Pod DNS Ayarları ve `hostAliases`

Alan	Davranış
<code>dnsPolicy: ClusterFirst</code>	Varsayılan; cluster alan adı dışındaki sorgular üst DNS'e iletilir ^[6]
<code>dnsPolicy: Default</code>	Pod, node'un DNS çözümleme yapılandırmasını devralır ^[6]
<code>dnsPolicy: None</code>	Kubernetes DNS varsayımlarını atlar; <code>dnsConfig</code> zorunludur ^[6]

```
spec:
  dnsPolicy: None
  dnsConfig:
    nameservers: ["10.96.0.10"]
    searches: ["net-lab.svc.cluster.local"]
    options:
      - name: ndots
        value: "2"
  hostAliases:
    - ip: "192.0.2.10"
      hostnames: ["legacy.internal"]
```

- `dnsConfig`, nameserver, search ve seçenekleri özelleştirir; örnekteki DNS IP'si cluster'a göre bulunmalıdır^[6]
- `hostAliases`, kubelet'in yönettiği Pod `/etc/hosts` dosyasına sabit ad ekler; DNS kaydı oluşturmaz. Dosyayı elle değiştirmek kalıcı değildir^[7]



Yakınlık ve trafik politikaları

Alan	Varsayılan	Değiştirildiğinde
<code>sessionAffinity</code>	<code>None</code>	<code>ClientIP</code> , aynı kaynak IP'yi mümkün olduğunca aynı endpoint'e yöneltir; varsayılan zaman aşımı 10.800 saniyedir ^[5]
<code>internalTrafficPolicy</code>	<code>Cluster</code>	<code>Local</code> , cluster içi istemciyi yalnızca aynı node'daki endpoint'lere yollar; yerel endpoint yoksa trafik düşer ^[5]
<code>externalTrafficPolicy</code>	<code>Cluster</code>	<code>Local</code> , dış trafiği node-local endpoint'lerle sınırlar; node'da yerel endpoint yoksa trafik iletilmez ^[5]

- `ClientIP` uygulama oturumu garantisi değildir; kaynak IP'yi anahtar aldığı için istemci IP'si değiştiğinde veya birçok istemci aynı IP'den görüldüğünde eşleşme de değişebilir^[5]
- `Local` politikası node'lar arası sıçramayı sınırlamak için bilinçli bir tercihtir; endpoint olmayan node'larda erişilebilirliği azaltabilir^[5]



Node IP'si

- Pod'ları çalıştıran makinelerin Pod ağından ayrı host adresidir^[2]
- Node yönetimi ve `NodePort` gibi node üzerinden girişlerde kullanılır



Erişim yöntemi seçimi

Gereksinim	İlk tercih
Cluster içi kalıcı erişim	DNS adı + <code>ClusterIP</code> Service
Kısa süreli yerel hata ayıklama	<code>kubectl port-forward</code>
Dış TCP/UDP erişimi	Ortama göre <code>NodePort</code> veya <code>LoadBalancer</code>
Dış HTTP(S) yönlendirme	Kurulu controller ile Ingress veya Gateway API



kubectl port-forward

Servis ya da iş yüklerinden kısa süreli port yönlendirme için kullanılır:

```
kubectl port-forward -n net-lab service/web 8080:80
```

- Seçilen türden bağımsız olarak bir Pod'u seçer, yalnız TCP taşır ve varsayılan olarak `localhost` üzerinde dinler^[8,9]
- İşlem terminali meşgul eder; `Ctrl+C` tüneli kapatır^[8]
- Seçilen yerel port (örnekte `8080`) doluysa `kubectl port-forward -n net-lab service/web :80` ile boş yerel portu otomatik seçilebilir; istekler için komutun yazdırdığı port kullanılır^[8,9]



Ingress

- Cluster içindeki bir servise dışarıdan ulaşım için (tipik olarak HTTP/S) kullanılan nesne tipi
- Cluster içinde hazır olarak gelmez, bir controller kurulmadan Ingress nesnesi oluşturulsa dahi fonksiyonel değildir
- `pathType` zorunludur. TLS sonlandırma için sertifika ve özel anahtar içeren Secret belirtilir; Ingress ile backend arasındaki trafik varsayılan olarak açık metindir^[10]
- Ingress API dondurulmuş ama kaldırılma planı yok, bir sonraki nesili Gateway API önerilir^[10]

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
spec:
  ingressClassName: nginx
  rules:
  - host: web.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web
            port:
              number: 80
```



Gateway API

- Gateway API, Ingress'te beraber olan değişik rolleri ayrı parçalara böler:

`GatewayClass` → altyapı/controller sınıfı → `Gateway` → listener ve adres → `HTTPRoute` → host/path ve backend^[11]

- `GatewayClass`, hangi controller'ın yöneteceğini; `Gateway`, listener'ları; `HTTPRoute`, eşleşme ve Service backend'lerini tanımlar^[11]
- Gateway varsayılan olarak yalnız kendi namespace'indeki Route'ları kabul eder; namespace'ler arası bağ için listener'da `allowedRoutes` açıkça yapılandırılır^[11]



Ingress ve Gateway API karşılaştırması

Boyut	Ingress	Gateway API
Kurulum	Yerleşik API; ayrıca controller gerekir	Eklenti CRD'leri ve uyumlu controller gerekir ^[10,11]
Sorumluluk	Tek Ingress kaynağında HTTP(S) kuralları	Sınıf, altyapı ve uygulama rotası ayrı kaynaklarda
İfade gücü	Host/path + uygulamaya özgü annotation'lar	Taşınabilir route bağlama, ağırlıklı backend ve daha geniş protokol modeli ^[11]



NetworkPolicy

NetworkPolicy, Pod'lar arası trafik kontrolü için kullanılır:

- Namespace'te hiçbir NetworkPolicy yoksa Pod ingress ve egress trafiği varsayılan olarak açıktır^[12]
- Bir politika Pod'u seçip sadece **Ingress** veya **Egress** yönünü kapsadığında Pod yalnız o yönde izole olur; iki yön bağımsızdır^[12]
- Pod A → Pod B bağlantısı için A'nın egress'i ve B'nin ingress'i varsa başarılı bağlantı için **ikisi de** izin vermelidir^[12]
- NetworkPolicy API kaynağının cluster'da bulunması çalışacağının kanıtı değil; CNI eklentisi NetworkPolicy'yi desteklemelidir^[12]

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
```

- Boş selector namespace'teki tüm Pod'ları seçer; izin kuralı bulunmadığı için iki yönde de yeni bağlantıya izin verilmez^[12]



Selector'lar, `ipBlock` ve DNS izni

Yer	Eşleşme
<code>spec.podSelector</code>	Politikanın uygulandığı, aynı namespace'teki Pod'lar
<code>from/to.podSelector</code>	Aynı namespace'teki eş Pod'lar
<code>namespaceSelector</code>	Etiketleri eşleşen namespace'lerdeki Pod'lar
<code>from/to.podSelector + namespaceSelector</code>	Pod ve Namespace koşullarının kesişimi
<code>ipBlock</code>	Tercihen cluster dışı CIDR; NAT'lama uygulamaya göre sonucu değiştirebilir

Bu seçiciler izin verilecek eşleri tanımlar; ayrı `from` / `to` maddeleri **veya** (OR), aynı madde içindeki koşullar **ve** (AND) mantığı taşır^[12]

```
egress:
- to:
  namespaceSelector:
    matchLabels: {kubernetes.io/metadata.name: kube-system}
  podSelector:
    matchLabels: {k8s-app: kube-dns}
  ports:
  - {protocol: UDP, port: 53}
  - {protocol: TCP, port: 53}
```



Sorun çözümleme

Bir erişim sorunuyla karşılaşılması durumunda birden fazla olasılık mevcut:

Sorun	Denetleme adımları
Ad çözülüyor	Kısa adın namespace'ini, FQDN'yi, <code>dnsPolicy</code> / <code>dnsConfig</code> ve CoreDNS'i denetle ^[6]
Service adı çözülüyor, bağlantı yok	Selector, EndpointSlice readiness, <code>port</code> / <code>targetPort</code> ve Service yönlendirme katmanını incele ^[3,4,5]
<code>port-forward</code> çalışıyor, Service VIP çalışmıyor	Uygulama ve seçilen Pod ayakta; Service yönlendirme katmanı ve NetworkPolicy yoluna odaklan ^[8,9]
Cluster içi çalışıyor, dışarıdan çalışmıyor	Service türü, node/bulut uygulaması, Ingress/Gateway controller ve dış ağ girişini doğrula ^[3,10,11]
NetworkPolicy sonrası kesildi	Kaynak egress'i ile hedef ingress'inin ikisini, DNS iznini ve CNI eklenti özellik uygunluğunu değerlendir ^[12]



Kaynaklar

1. Services, Load Balancing, and Networking: kubernetes.io/docs/concepts/services-networking/
2. Cluster Networking: kubernetes.io/docs/concepts/cluster-administration/networking/
3. Service: kubernetes.io/docs/concepts/services-networking/service/
4. EndpointSlices: kubernetes.io/docs/concepts/services-networking/endpoint-slices/
5. Virtual IPs and Service Proxies: kubernetes.io/docs/reference/networking/virtual-ips/
6. DNS for Services and Pods: kubernetes.io/docs/concepts/services-networking/dns-pod-service/
7. Adding Entries to Pod /etc/hosts with HostAliases: kubernetes.io/docs/tasks/network/customize-hosts-file-for-pods/
8. Use Port Forwarding to Access Applications in a Cluster: kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/
9. kubectl port-forward: kubernetes.io/docs/reference/kubectl/generated/kubectl_port-forward/
10. Ingress: kubernetes.io/docs/concepts/services-networking/ingress/
11. Gateway API: kubernetes.io/docs/concepts/services-networking/gateway/
12. Network Policies: kubernetes.io/docs/concepts/services-networking/network-policies/
13. kubectl expose: kubernetes.io/docs/reference/kubectl/generated/kubectl_expose/
14. kubectl get: kubernetes.io/docs/reference/kubectl/generated/kubectl_get/